

TapTech Sentinel: An AI-Powered Kubernetes Ingress Security Operator

Architecture, Threat Detection Pipeline, and Design Decisions

Abstract

TapTech Sentinel is an AI-powered Kubernetes Ingress security operator that watches NGINX Ingress access logs in real-time, detects threats through pattern analysis and LLM-based classification, and enforces IP blocks by patching NGINX ConfigMaps—all without requiring manual intervention or deep Kubernetes expertise. This paper describes the architecture of the operator pipeline, from log ingestion through enforcement; the design decision to use Kubernetes Custom Resource Definitions (CRDs) as the primary state model rather than an external datastore; and the automated threat reporting system that delivers daily security intelligence to operations teams.

1. The Problem

Kubernetes Ingress controllers sit at the boundary between the public internet and internal services, making them a natural target for automated scanners, credential stuffing bots, vulnerability probes, and denial-of-service attacks. Defending this boundary in a Kubernetes environment presents several challenges:

Manual log review does not scale. NGINX Ingress pods can produce thousands of log entries per minute across multiple replicas. Operators who rely on periodic manual inspection inevitably miss attack patterns until damage is done.

Scripted alerting is brittle. Shell scripts and cron jobs that parse logs and fire alerts require constant maintenance as attack patterns evolve. They typically lack context—flagging individual requests without correlating them into coherent threat profiles.

Enforcement requires Kubernetes expertise. Blocking a malicious IP in an NGINX Ingress ConfigMap requires understanding ConfigMap structure, the `server-snippet` annotation, NGINX deny-list syntax, and `kubectl` patching semantics. This puts response capability solely in the hands of platform engineers, excluding security analysts and operations staff who may identify threats first.

No audit trail. Ad-hoc `kubectl edit` commands to block IPs leave no structured record of what was blocked, when, why, or by whom. Forensic review after an incident becomes a matter of correlating shell histories and Slack messages.

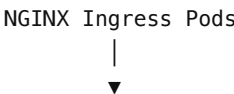
Static blocklists go stale. External threat intelligence feeds (Spamhaus DROP, abuse.ch, etc.) require periodic fetching, parsing, and application—another manual process that drifts out of date.

TapTech Sentinel addresses these problems by automating the full cycle—detection, classification, enforcement, reporting, and conversational management—within a Kubernetes-native architecture.

2. Architecture Overview

Sentinel is structured as a Java 21 Spring Boot application built on the Java Operator SDK (JOSDK) and the Fabric8 Kubernetes Client. It runs as a single deployment inside the cluster (or externally during development) and interacts with the Kubernetes API server to watch logs, manage CRDs, and patch ConfigMaps.

The high-level data flow is:





Three Custom Resource Definitions anchor the system:

- **IPBlockRule** — An IP address or CIDR range to block, with duration, threat level, reason, and source attribution. Reconciled into NGINX deny-list directives.
- **ThreatReport** — A detected threat record capturing the source IP, threat level, category, request count, targeted paths, user agents, and optional AI analysis. Triggers real-time alerts.
- **BlockList** — An external blocklist URL with format, refresh interval, and sync status. Reconciled by fetching the list and creating corresponding IPBlockRules.

The JaiClaw AI orchestration framework provides a conversational interface over Telegram and an interactive shell CLI, allowing operators to query threat history, ban or unban IPs, and generate reports through natural language commands.

3. The Operator Pipeline

3.1 Log Ingestion

The `NginxLogWatcher` implements Spring's `SmartLifecycle` interface and runs on a Java 21 virtual thread. On startup, it discovers NGINX Ingress pods by querying the configured namespace with a label selector (default: `app.kubernetes.io/name=ingress`), filtering for pods in `Running` phase.

Log streaming uses the Fabric8 Kubernetes Client's `watchLog()` API:

```
client.pods()
    .inNamespace(namespace)
    .withName(podName)
    .tailingLines(0)
    .watchLog();
```

The `tailingLines(0)` parameter starts reading from the current tail of the log stream, avoiding replay of historical entries on startup. The returned `LogWatch` provides an `InputStream` that is wrapped in a `BufferedReader` for line-by-line

processing.

Reconnection is automatic: if the log stream is interrupted (pod restart, network partition), the watcher retries after 10 seconds. If the pod cannot be found (e.g., during a rolling update), it retries after 30 seconds. The virtual thread model keeps this I/O-bound loop efficient without consuming a platform thread.

3.2 Log Parsing

Each log line passes through `NginxAccessLogParser`, which applies a compiled regular expression matching the NGINX combined log format. The pattern extracts 10 capture groups into an `AccessLogEntry` record with 11 fields: `clientIp`, `timestamp`, `method`, `path`, `protocol`, `statusCode`, `bodyBytesSent`, `referer`, `userAgent`, `upstreamAddr`, and `requestId`. Timestamps are parsed from NGINX's `dd/MMM/yyyy:HH:mm:ss Z` format into `Instant` values. Lines that do not match the pattern are silently skipped—this handles NGINX startup messages and other non-access-log output.

3.3 Pattern Detection

The `PatternDetector` maintains three categories of attack signatures:

Scanner user-agent signatures (15 patterns). Case-insensitive regex patterns matching known scanning tools: `zgrab`, `sqlmap`, `nikto`, `nmap`, `masscan`, `dirbuster`, `gobuster`, `wpscan`, `acunetix`, `nessus`, `openvas`, `burpsuite`, `python-requests`, `Go-http-client`, and `libwww-perl`. These are stored in a `CopyOnWriteArrayList<Pattern>` for thread-safe concurrent access.

Vulnerability probe paths (28 paths). Known paths that indicate reconnaissance or exploitation attempts: credential files (`/.env`, `/.aws/credentials`, `/.htpasswd`), version control metadata (`/.git/config`, `/.git/HEAD`), CMS attack surfaces (`/wp-login.php`, `/wp-admin`, `/xmlrpc.php`, `/wp-config.php.bak`), administration panels (`/phpmyadmin`, `/pma`, `/administrator`, `/solr/admin`, `/console`), Spring Boot endpoints (`/actuator`, `/actuator/env`), database dumps (`/backup.sql`, `/dump.sql`, `/db.sql`), server metadata (`/server-status`, `/server-info`), Kubernetes API probes (`/api/v1/pods`), and legacy attack vectors (`/cgi-bin/`, `/shell`, `/remote/fgt_lang`). These are stored in a `ConcurrentHashMap`-backed key set.

Structural regex patterns (2 compiled patterns). A path traversal detector (`\.\./`) and a SQL injection detector covering `UNION SELECT`, `OR 1=1`, tautology injections, comment-based attacks, and `DROP` statement injection.

All three categories are extensible at runtime through `addScannerUaPattern()`, `addVulnerabilityPath()`, and `addCustomPattern()` methods, allowing new signatures to be deployed without restarts.

3.4 Rate Tracking

The `RateTrackerStrategy` interface provides a sliding-window rate counter per IP address. Two implementations are available:

InMemoryRateTracker uses a `ConcurrentHashMap<String, Queue<Instant>>` where each IP maps to a `ConcurrentLinkedQueue` of request timestamps. On each request, the current timestamp is appended and entries older than the window are evicted. This is the default implementation when Redis is not configured.

RedisRateTracker uses Redis sorted sets (ZSETs) with epoch-millisecond scores. On each request, the timestamp is added to the set, entries below the window cutoff are removed via `ZREMRANGEBYSCORE`, and the set's TTL is set to 2× the rate window for automatic cleanup. This implementation supports multi-replica deployments where rate state must be shared.

The default threshold is 100 requests per minute with a 1-minute sliding window, configurable through application properties.

3.5 Threat Analysis

The `ThreatAnalyzer` combines pattern and rate signals to determine a threat level:

Condition	Threat Level	Category
Pattern match AND rate exceeded	CRITICAL	From pattern match
Rate > 2× threshold (no pattern)	HIGH	DDoS
Rate exceeded (no pattern)	HIGH	UNKNOWN
Pattern match only	MEDIUM	From pattern match

A 5-minute debounce period prevents duplicate reports for the same IP. Debounce state is maintained in Redis (when available) or an in-memory `ConcurrentHashMap<String, Instant>` .

The analyzer tracks per-IP path sets and user agent sets (`ConcurrentHashMap` of `Set` values), including the top 20 targeted paths and top 5 user agents in `ThreatReport` CRDs for forensic context.

When a threat reaches HIGH or CRITICAL and auto-blocking is enabled (the default), the analyzer creates an `IPBlockRule` CRD with source `"ai-analysis"` , a 24-hour default duration, and a reason string that includes the threat category, request count, and top targeted paths. This CRD creation triggers the enforcement pipeline through the reconciler.

3.6 AI Classification

For ambiguous patterns where rule-based detection is insufficient, the `AiDecisionEngine` invokes Claude via Spring AI. The engine constructs a structured prompt containing the source IP, request count, time window, targeted paths, and user agents, asking the model to classify the threat into one of nine categories (SCANNER, BRUTE_FORCE, CVE_PROBE, PATH_TRAVERSAL, SQL_INJECTION, CREDENTIAL_STUFFING, BOT, DDoS, UNKNOWN), assign a threat level, provide a confidence score, recommend a block duration, and supply a brief analysis.

The response is parsed by scanning for enum value names, making the integration resilient to variations in model output formatting. When the LLM is unavailable, fallback heuristics apply: requests exceeding 200 in the window are rated HIGH; known scanner user agents elevate the assessment; and confidence is set to 0.3 to signal reduced certainty.

The `ThreatAssessment` record returned includes 8 fields: `sourceIp` , `threatLevel` , `category` , `analysis` , `recommendedAction` , `recommendedBlockDuration` , `confidence` , and `indicators` .

4. Why CRDs: An Architectural Decision Record

One of the most consequential design decisions in Sentinel was choosing Kubernetes Custom Resource Definitions as the primary state model for block rules, threat reports, and blocklist configurations. The alternative—a Redis datastore, a SQL database, or a custom persistence layer—would have been a natural choice for many application architectures. In Sentinel's case, CRDs are the superior fit for several reasons.

4.1 Native Kubernetes Integration

CRDs are first-class Kubernetes objects. They inherit RBAC-based access control, admission webhooks, API versioning, and audit logging from the platform itself. An operator can run `kubectl get ipblockrules` to list all active blocks, `kubectl describe ipblockrule suspicious-ip-42` to inspect a specific rule's status, or `kubectl delete ipblockrule temporary-block` to remove one—using the same tools and permissions model they use for every other Kubernetes resource. No additional CLI, dashboard, or API endpoint is needed.

4.2 The Reconciliation Model

The Java Operator SDK watches CRDs and invokes reconciler methods on every create, update, or delete event. This is the idiomatic Kubernetes controller pattern: declare the desired state, and the reconciler drives actual state to match. If a

reconciliation fails (network error, temporary API unavailability), JOSDK automatically retries with exponential backoff. Eventual consistency is guaranteed by the platform.

With a Redis-based approach, the application would need to implement its own change-detection mechanism—polling for changes, maintaining a watch stream over a pub/sub channel, or building a custom event system. Each of these introduces complexity and failure modes that the Kubernetes control plane already handles.

4.3 No Additional Infrastructure

CRDs are stored in etcd, which is already running as part of the Kubernetes control plane. Sentinel requires no additional database to deploy, configure, monitor, back up, or secure. Adding Redis (or any external datastore) as the primary state store would mean operating another stateful component: provisioning persistent volumes, configuring authentication, setting up replication for high availability, monitoring memory usage, and maintaining backup procedures.

For a security-critical system, every additional infrastructure component is another attack surface, another credential set to manage, and another failure mode to handle. CRDs eliminate this entire class of operational burden.

4.4 Declarative State and GitOps Compatibility

Because CRDs are standard Kubernetes YAML, they integrate naturally with GitOps workflows. An IPBlockRule can be defined in a Git repository, reviewed in a pull request, and applied through ArgoCD or Flux alongside every other cluster resource. Pre-approved blocks can be committed as part of a security baseline, and drift detection ensures they remain enforced.

Redis state, by contrast, is opaque to the Kubernetes ecosystem. It cannot be version-controlled, code-reviewed, or managed through standard deployment pipelines without building custom tooling.

4.5 Built-in Audit Trail

Every CRD mutation is recorded in the Kubernetes API audit log with a timestamp, actor identity (user or service account), the operation performed, and the before/after resource state. For a security operations tool, this audit trail is not optional—it is essential for incident response, compliance, and accountability. Building equivalent audit capability over a Redis datastore would require implementing a write-ahead log, serializing state changes, and correlating them with actor identity—functionality that the Kubernetes API server provides inherently.

4.6 The Hybrid Approach

Sentinel does not reject Redis entirely. It adopts a hybrid strategy that uses each data store for what it does best:

- **CRDs** for durable, auditable, low-frequency state: block rules (created on threat detection, deleted on expiry), threat reports (created per incident), and blocklist configurations (created once, refreshed periodically).
- **Redis** (optional) for high-frequency ephemeral data: per-IP request rate counters (updated on every log line), deduplication keys (checked and set on every threat evaluation), and block-set caching for fast lookup.

This separation ensures that the high-write-volume rate tracking does not burden the Kubernetes API server, while the security-critical state that requires auditability, RBAC, and GitOps compatibility lives where it belongs—in the cluster's native resource model.

5. Enforcement: From CRD to NGINX

5.1 IPBlockRuleReconciler

When an `IPBlockRule` CRD is created or updated, the reconciler:

1. **Computes expiry.** If the status lacks an `expiresAt` timestamp and the spec includes a `duration` (supporting `30m`, `24h`, `7d` formats), it calculates the expiry from the resource's creation timestamp plus the duration.

2. **Aggregates active rules.** Lists all `IPBlockRule` resources across namespaces, filtering out expired entries.
3. **Enforces.** Passes the full active rule set to `NginxDenyListEnforcer`.
4. **Syncs to Redis.** If the optional `RedisBlockSyncService` is available, pushes the active block set for fast lookup by other components.
5. **Updates status.** Sets `enforced = true` and records `enforcedAt` as an ISO-8601 timestamp.

On deletion, the reconciler re-enforces with the remaining active rules, ensuring the deny-list accurately reflects the current state.

5.2 NginxDenyListEnforcer

The enforcer operates on the NGINX Ingress Controller's ConfigMap, specifically the `server-snippet` key that NGINX Ingress supports for injecting arbitrary server-block directives.

The critical design constraint is coexistence: the ConfigMap's `server-snippet` may contain user-defined directives that must not be disturbed. Sentinel uses a marker-based merge strategy:

```
# --- Sentinel deny-list (auto-managed) ---
deny 1.2.3.4;
deny 10.0.0.0/8;
# --- Sentinel deny-list (auto-managed) ---
```

The `mergeSentinelSnippet()` method locates these markers in the existing snippet and replaces only the content between them. If no markers exist, the block is appended. Deny directives are sorted and deduplicated. Each active rule produces a `deny <effectiveTarget>;` directive, where `effectiveTarget()` returns the CIDR if set, otherwise the IP.

NGINX Ingress Controller watches its ConfigMap for changes and automatically reloads the NGINX configuration—no pod restart or manual reload is required. This means enforcement is effective within seconds of the ConfigMap patch.

5.3 External Blocklist Integration

The `BlockListReconciler` manages `BlockList` CRDs, each pointing to an external threat intelligence feed. On reconciliation, the `BlockListFetcher` downloads the list via HTTP (with 30-second connect and 60-second request timeouts) and parses it according to the configured format:

- **CIDR_COMMENTS** — Lines with optional `; comment` suffix (Spamhaus DROP format)
- **PLAIN** — One IP or CIDR per line
- **CSV** — First column extracted, comments skipped

For each entry, the reconciler creates an `IPBlockRule` CRD with source attribution (`blocklist:{name}`) and a configurable default duration (7 days if unspecified). Stale rules from previous syncs are deleted, ensuring the rule set reflects the current state of the external feed.

6. Threat Report Generation

6.1 The Problem It Solves

Security teams need regular visibility into the threat landscape without manually querying the cluster. They need to know: How many threats were detected? What are the most targeted paths? Which IPs are repeat offenders? Are there unblocked threats that require attention?

6.2 Scheduling via JaiClaw Cron Integration

Sentinel integrates with JaiClaw's `CronJobManagerService` to schedule daily report generation. The `ThreatReportCronConfig` registers a cron job (default: `0 6 * * *`, daily at 6:00 AM Eastern) that fires a prompt to the AI agent: *"Call sentinel_generate_report to generate and send the daily threat report."* The agent invokes the report generation tool, which produces and delivers the report via SMTP.

This approach requires no external cron daemon and no Kubernetes CronJob manifest—the scheduling lives within the operator process, managed through the same JaiClaw framework that handles conversational interactions.

6.3 Report Content

The `ThreatReportGenerator` queries all `ThreatReport` and `IPBlockRule` CRDs and computes the following aggregations:

- **Threat counts by level** — CRITICAL, HIGH, MEDIUM, LOW tallied via `EnumMap<ThreatLevel, Long>`
- **Threat counts by category** — All nine categories (SCANNER through UNKNOWN) ranked by frequency
- **Top 10 offending IPs** — Source IPs grouped and sorted by incident count
- **Top 10 targeted paths** — Paths flattened from all threat reports, grouped and counted
- **Recent blocks (last 24 hours)** — `IPBlockRules` created within the reporting window
- **Unblocked IPs** — Threat source IPs with no corresponding active block rule, surfaced for operator action

The output is generated in both HTML and plain text formats. The HTML version includes:

1. **Executive summary** with total threat count, active blocks, and new blocks in the last 24 hours
2. **Threat level breakdown** with color-coded indicators (CRITICAL: red, HIGH: orange, MEDIUM: yellow, LOW: green)
3. **Top threat categories** ranked by frequency
4. **Top offending IPs** and **top targeted paths** tables
5. **Per-threat detail table** with IP, BLOCKED/UNBLOCKED status badges, level, category, request count, paths, and user agents
6. **Quick Actions section** with a pre-formatted Telegram batch ban command for unblocked IPs, ready to copy and execute
7. **Recent blocks** and **all active blocks** with expiry information

Reports are delivered via SMTP to configured recipients, providing a daily security briefing without requiring cluster access.

7. Event-Driven Alerting

Real-time alerting complements the daily report. The `ThreatReportReconciler` uses Spring's `ApplicationEventPublisher` to fire a `ThreatReportCreatedEvent` whenever a new `ThreatReport` CRD is created. An idempotency guard (`status.alertSent`) ensures each report triggers at most one alert.

The JaiClaw gateway subscribes to these events and pushes formatted alerts to Telegram, where authorized operators can review the threat details and respond conversationally—querying history, issuing ban commands, or dismissing false positives—without opening a terminal or constructing `kubectl` commands.

Telegram security is enforced through JaiClaw's `TelegramUserIdFilter` (allowlisting specific Telegram user IDs) and `UserRateLimiter`, ensuring that only authorized personnel can interact with the bot.

8. Conclusion

TapTech Sentinel demonstrates that Kubernetes-native architecture, combined with AI-powered analysis, can automate the full ingress security lifecycle—from raw log line to enforced deny-list entry—in a way that is auditable, operationally lightweight, and accessible to non-Kubernetes specialists.

The choice of CRDs as the state model is not incidental; it is foundational. By representing block rules, threat reports, and blocklist configurations as first-class Kubernetes resources, Sentinel inherits the platform's reconciliation guarantees, RBAC, audit logging, and GitOps compatibility without building any of it from scratch. The hybrid use of Redis for high-frequency ephemeral data preserves API server performance while keeping durable state where it belongs.

The automated reporting pipeline—from cron-triggered AI agent invocation through CRD aggregation to formatted email delivery—provides daily security intelligence without custom infrastructure. Real-time Telegram alerts give operators immediate visibility and conversational control over the system's response.

The result is a security operator that monitors continuously, responds in sub-second time frames, maintains a complete audit trail, and puts threat management within reach of any authorized team member through natural language interaction.